

WTFOS

YouTube Analytics & Episode Intelligence

Developer Requirements

Functional + technical brief for the YouTube module of WTFOS

Primary Objective

Build a first-party YouTube intelligence layer inside WTFOS that connects directly to the WTF YouTube channel, ingests video and analytics data, stores historical performance, maps assets to podcast episodes, analyzes comments and transcripts, and powers episode-level AI insights.

Audience: WTFOS developer / engineering team

Phase focus: YouTube only

Implementation principle: WTFOS owns the developer app. WTF channel admins authorize access through Google OAuth. Do not use shared passwords.

1. What We Need Built

- WTFOS must have a “Connect YouTube” integration using Google OAuth 2.0.
- After authorization, WTFOS should automatically ingest the complete WTF YouTube video catalogue and keep it synchronized.
- Use the YouTube Data API v3 for channel/video metadata and public/basic statistics.
- Use the YouTube Analytics API for authenticated performance reporting by video and date range.
- Store daily historical metrics in our own database rather than relying on live API calls from the dashboard.
- Create an internal Episode entity so a full podcast, Shorts, teasers, clips and related assets can all belong to the same episode.
- Pull comments for each video and store them for search and AI analysis.
- Ingest or generate episode transcripts, segment them by timestamp, topic and speaker where possible.
- Create an AI intelligence layer that combines analytics + comments + transcript + internal episode metadata.
- Expose the data through a dashboard and an “Ask WTFOS” conversational interface.

Important

We are not trying to clone YouTube Studio. The goal is to create a strategic content intelligence system around the WTF podcast: episodes, guests, topics, clips, audience reaction, growth and repeatable learnings.

2. Access & Authentication

The WTF team should not provide us with its Google/YouTube password or a manually shared API key. WTFOS should register its own Google Cloud application and request read-only authorization from an authorized WTF channel account.

1. Create a Google Cloud project dedicated to WTFOS.
2. Enable YouTube Data API v3.
3. Enable YouTube Analytics API.
4. Configure the OAuth consent screen and production app details.
5. Create an OAuth 2.0 Client ID / Client Secret for the WTFOS web application.
6. Add production and staging redirect URIs, e.g. <https://<wtfos-domain>/auth/youtube/callback>.
7. Add a Connect YouTube button inside WTFOS Settings > Integrations.
8. Authorized WTF admin signs in with Google and approves the requested scopes.
9. Backend securely stores access and refresh tokens; tokens must never be exposed to the frontend.
10. Implement token refresh, revocation handling and a Disconnect/Reconnect flow.

Security requirement

Encrypt OAuth credentials/tokens at rest, keep secrets in a secure secret manager/environment, never log tokens, and use the minimum read-only scopes required.

3. Recommended Architecture

Layer	Responsibility
YouTube Connector	OAuth, Data API, Analytics API, comment ingestion
Sync Workers	Scheduled fetches, retries, backfills, incremental updates
Data Warehouse / DB	Raw payloads + normalized metrics + episode relationships
Intelligence Engine	Comments, transcript, topics, benchmarks, AI summaries
WTFOS UI/API	Dashboard, episode pages, filters, search, Ask WTFOS

Recommended data flow: YouTube APIs → WTFOS connector → scheduled sync → normalized database → episode intelligence layer → dashboard / AI.

4. Data We Need From YouTube

Dataset	Source	Examples	Frequency
Channel metadata	YouTube Data API	channel ID, title, subscriber/video counts where available	Daily
Video catalogue	YouTube Data API	video ID, title, description, publish time, duration, thumbnail, playlist	Initial backfill + hourly/daily incremental

Video statistics	YouTube Data API	views, likes, comments and other available statistics	Several times/day or daily
Authenticated analytics	YouTube Analytics API	views, watch time, average view duration, subscribers gained/lost, shares, geographies/devices/sources where supported	Daily + recent-content refresh
Comments	YouTube Data API	top-level comments, replies, author display data, likes, publish/update time	Frequent for new videos; slower for archive
Transcript	Transcript/caption source or approved internal transcription process	timestamped spoken content	Once per new long-form video; reprocess if edited

5. Core Data Model

The database should keep YouTube-native entities separate from WTFOS business entities. The key requirement is that a YouTube video can be linked to an internal Episode.

channels

id, youtube_channel_id, title, connected_account, oauth_status, last_synced_at

videos

id, youtube_video_id, channel_id, title, description, published_at, duration_seconds, content_type (full_episode / short / teaser / clip / other), episode_id nullable, raw_metadata_json

episodes

id, episode_number, title, guest_id, guest_name, category, topics/tags, full_episode_video_id, published_at, status

video_daily_metrics

video_id, date, views, watch_minutes, average_view_duration, likes, comments_count, shares, subscribers_gained, subscribers_lost, other supported metrics, raw_analytics_json

comments

id, youtube_comment_id, video_id, parent_comment_id nullable, author_display_name, text, like_count, published_at, updated_at, detected_timestamp_seconds nullable, sentiment nullable, topic_cluster nullable, embedding/vector reference

transcript_segments

id, video_id, start_seconds, end_seconds, speaker nullable, text, topic nullable, embedding/vector reference

guests

id, name, industry/category, notes, external IDs/metadata as required

Historical storage is mandatory

Do not overwrite the latest view count as the only source of truth. Store date-based analytics so WTFOS can calculate Day 1 / Day 7 / Day 30 / lifetime performance, growth velocity, decay curves and fair episode comparisons.

6. Episode-Level Intelligence

An Episode is a WTFOS entity, not a YouTube API object. The developer should create a manual and/or rule-based mapping interface to associate videos with an episode.

- One episode can contain one full podcast plus any number of Shorts, clips, teasers or follow-up uploads.
- The full episode and each derivative asset must retain its own YouTube metrics while also rolling up into episode totals.
- Episode metadata should support guest, category, topic tags, publish date, episode number and internal notes.
- Admin UI should allow re-linking or correcting asset-to-episode relationships without touching YouTube.

Required episode calculations

- Full episode views / watch time / average view duration
- Subscribers gained and lost
- Day 1, Day 7, Day 30 and lifetime performance
- View velocity and growth curve
- Episode vs channel baseline
- Episode vs previous N episodes
- Total views across all clips/Shorts linked to that episode
- Best-performing derivative asset
- Guest-level and topic-level rollups

7. Comment Intelligence

Comments should be treated as a first-class dataset. WTFOS must be able to show comments for a specific video/episode and use them as input for AI analysis.

Raw comment capabilities

- Fetch and paginate top-level comments.
- Fetch replies where available/needed.
- Store comment text, author display name, likes and timestamps.
- Incrementally fetch new/updated comments after the initial backfill.
- Allow episode/video filtering, search and sorting by newest / most liked / most relevant.

Derived audience intelligence

- Overall sentiment and sentiment trends.
- Topic/theme clustering.
- What viewers loved most.
- What viewers disliked or criticized.
- Recurring questions.
- Requests for Part 2 / repeat guests.
- Future guest requests.
- Production/audio/video complaints.
- Quotes or phrases that viewers repeatedly reference.
- Compare comment themes and sentiment across episodes.

Timestamp intelligence

Detect YouTube-style timestamps inside comments (e.g. 37:42 or 1:14:32), normalize them to seconds, cluster nearby references, and join them with the transcript segment at that moment. This should produce “most talked-about moments” and candidate clips.

8. Transcript & Content Intelligence

For every full-length episode, WTFOS should maintain a timestamped transcript. The transcript should be segmented into searchable blocks and enriched with topics/themes. Speaker attribution is desirable where technically reliable.

- Timestamped transcript segments
- Topic extraction by segment
- Named entity extraction: people, companies, products, locations where useful
- Semantic embeddings for retrieval
- Potential clip-worthy moments
- Join comment timestamps to transcript context
- Enable natural-language search across episodes

9. AI / “Ask WTFOS” Layer

The AI layer should not query YouTube live for every question. It should answer from the synchronized WTFOS database and retrieval index, using the relevant analytics, comments, transcript and episode metadata.

- What did the audience like most about Episode 87?
- What are viewers criticizing in the latest episode?
- Which moments are most frequently referenced in comments?
- Which clips from this episode performed best?
- Which guests generated the highest subscriber conversion?
- Which topics produce high watch time but lower views?
- Compare the latest episode with the previous 10 episodes.
- Are viewers asking for this guest to return?
- Which future guests are most frequently requested?
- What recurring production complaints are appearing in comments?

AI response requirement

Where possible, the AI should show the evidence behind an answer: relevant episode/video, metric period, supporting comment clusters and transcript timestamps. Avoid unsupported “insight” text that cannot be traced back to stored data.

10. WTFOS YouTube UI Requirements

A. YouTube Overview

- Channel-level KPIs
- Views and watch time over selected period
- Subscriber growth
- Top videos
- Top episodes
- Long-form vs Shorts performance
- Recent growth anomalies / highlights

B. Episodes

- Searchable/filterable episode list
- Episode KPI summary
- Day 1 / 7 / 30 / lifetime curves
- Full episode + linked assets
- Episode benchmark vs channel baseline
- Guest/category/topic tags

C. Audience Voice

- Comment count
- Sentiment
- Top themes
- Top positive/negative themes
- Most liked/relevant comments
- Repeated questions
- Guest requests
- Timestamp-mentioned moments

D. Transcript / Moments

- Search transcript
- Browse timestamped topics
- Top discussion moments
- Potential clips
- Jump from a comment timestamp to transcript context

E. Ask WTFOS

- Natural-language interface scoped to YouTube
- Optional filters: episode, guest, date range, topic
- Answers grounded in our stored data

11. Sync & Processing Logic

- Initial backfill: fetch the channel catalogue and historical analytics within API-supported limits.
- Incremental catalogue sync: detect newly published videos automatically.
- Recent-content refresh: sync new episodes/Shorts more frequently for the first few days after publication.
- Historical refresh: reduce frequency for older videos while still updating lifetime metrics.
- Comments: aggressively sync recent content; use a slower cadence for older videos.
- Every sync job must be idempotent, retry-safe and observable.
- Store last successful sync time and API errors per resource/job.
- Use queues/workers rather than performing large API fetches in a user-facing web request.
- Respect YouTube API quotas and use batching/caching wherever supported.
- Keep raw source payloads where practical so we can re-normalize without a complete refetch.

12. Admin / Operations Requirements

- Connection status and last sync timestamp
- Reconnect/reauthorize YouTube
- Force sync for a specific video/episode
- Episode-to-video mapping editor
- Manual edit for guest/topic/category metadata
- Job/error dashboard for failed syncs

- Basic API quota monitoring
- Delete/disconnect workflow and token revocation
- Role-based access: only admins can manage integrations

13. Recommended Build Phases

Phase 1 - Core Connection & Data Foundation

OAuth; channel/video ingestion; Data API + Analytics API; normalized DB; daily historical metrics; basic YouTube overview.

Phase 2 - Episode Model

Episode entities; map full episodes/Shorts/clips; episode pages; 1/7/30/lifetime comparisons; guest/topic metadata.

Phase 3 - Comments & Audience Voice

Comment ingestion; search; sentiment; themes; questions; guest requests; timestamp parsing.

Phase 4 - Transcript & Semantic Intelligence

Timestamped transcript ingestion; topic segmentation; embeddings; comment-to-transcript linking; moment intelligence.

Phase 5 - Ask WTFOS

RAG/conversational interface across metrics + comments + transcript; grounded evidence; benchmarks and strategic insights.

14. Phase 1 Acceptance Criteria

- ☐ An authorized WTF admin can connect the WTF YouTube channel through Google OAuth without sharing credentials.
- ☐ WTFOS can list the channel's existing videos and automatically detect new uploads.
- ☐ WTFOS stores video metadata and daily analytics in its own database.
- ☐ An admin can open any video and view its WTFOS-stored performance history.
- ☐ An admin can create/edit an Episode and attach YouTube videos to it.
- ☐ WTFOS can calculate Day 1 / 7 / 30 / lifetime performance where sufficient history is available.
- ☐ Comments for a video can be ingested and displayed.
- ☐ All sync jobs expose status, last success and errors.
- ☐ Dashboard views do not depend on a live YouTube API call for normal page load.
- ☐ Credentials and OAuth tokens are not exposed to the client/browser or application logs.

15. What the Developer Should Deliver Back

- Proposed backend architecture and tech stack for the connector/sync layer
- Google Cloud/OAuth setup checklist and required scopes

- Final database schema / migrations
- API endpoint specification for the WTFOS frontend
- Sync schedule and quota strategy
- Comment ingestion strategy
- Transcript ingestion strategy
- AI/RAG design for Ask WTFOS
- Security/secret-management plan
- Effort estimate broken down by the phases above

North-star product definition

WTFOS YouTube should answer not only “how many views did this video get?” but “which episode, guest, topic, moment and audience reaction created the most value for WTF - and why?”

Prepared as a developer handoff brief for the WTFOS YouTube analytics module.